



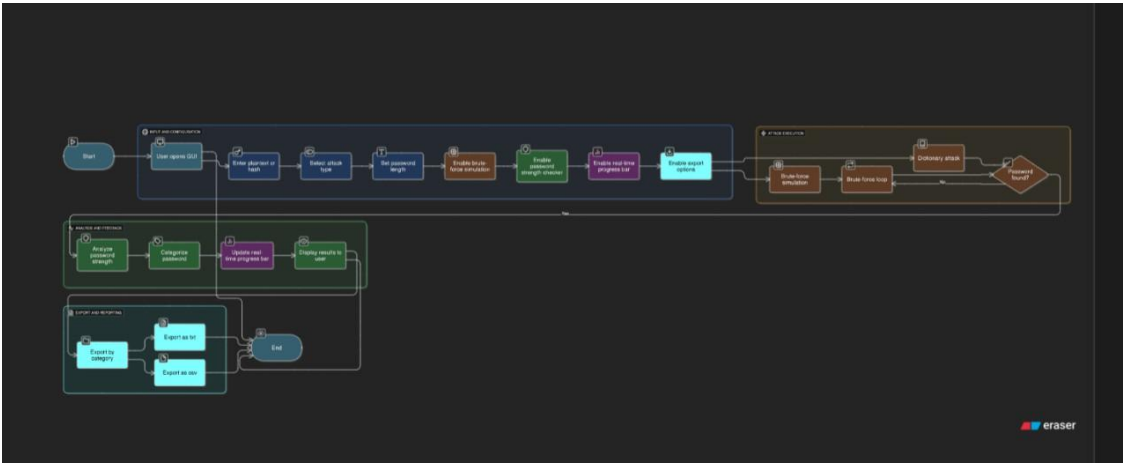
DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Visvesvaraya Technological University (VTU), Belagavi,
Approved by AICTE and UGC, Accredited by NAAC with 'A' grade & ISO 9001-2015 Certified Institution)

Shavige Malleshwara Hills, Kumaraswamy Layout, Bengaluru - 560 111. India

DEPARTMENT	ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING				
Semester: VII	Course Title: Data Security & Privacy Lab			Course Code: 22AI73	
PROGRAM QUESTION	2) Build python code for Dictionary Attack for plaintext and hash value. a) Add brute-force simulation in this GUI? b) Add password strength checker (like weak/medium/strong)? c) Export analyzed passwords by category? d) Support longer passwords (4+ characters)? e) Export analysis report as .txt or .csv? f) Add real-time progress bar for brute-force?				
STUDENT NAMES	Udayaraj		USN	1DS22AI057	
LAB MARKS	Program Execution (10)	Source Code (10)	Result (5)	Innovation/ Extra Features (5)	Total
TOOLS USED	Cursor				
MAP COURSE OUTCOME	CO1				
MAP PROGRAM OUTCOME	PO2, PO3, PO5, PO9, PO11				
MAP PROGRAM SPECIFIC OUTCOME	PSO1, PSO2, PSO3				

UML / SEQUENCE DIGRAM



COURSE COORDINATOR

Dr ARUNA M G & Prof ENSTEIH SILVIA

Code for Program 2:

- 2) Build python code for Dictionary Attack for plaintext and hash value.
- a) Add brute-force simulation in this GUI?
- b) Add password strength checker (like weak/medium/strong)?
- c) Export analyzed passwords by category?
- d) Support longer passwords (4+ characters)?
- e) Export analysis report as .txt or .csv?
- f) Add real-time progress bar for brute-force?

```

import streamlit as st, hashlib, pandas as pd, time, re
from itertools import product
from typing import List, Optional

st.set_page_config(page_title="🔒 Password Tool", page_icon="🔑", layout="wide")

# ----- Password Strength -----
def check_strength(p: str):
    score, fb = 0, []
    if len(p) >= 12: score, fb = score+3, fb+["✓ 12+ chars"]
    elif len(p) >= 8: score, fb = score+2, fb+["✓ 8-11 chars"]
    elif len(p) >= 4: score, fb = score+1, fb+["✓ 4-7 chars"]
    else: fb += ["X Too short"]

    for regex, msg, val in [(r'[A-Z]', "✓ Upper", 1), (r'[a-z]', "✓ Lower", 1),

```

```

(r'[0-9]','✓ Number',1),(r'^A-Za-z0-9','✓ Special',2)]:

if re.search(regex,p): score, fb = score+val, fb+[msg]
if any(x in p.lower() for x in ['123','abc','qwerty','password','admin']):
    score-=1; fb+=[" ■ Common pattern"]
return ("Strong" if score>=7 else "Medium" if score>=4 else "Weak"), score, fb

# ----- Dictionary / Brute Force -----
HASHES = {'md5':hashlib.md5,'sha1':hashlib.sha1,'sha256':hashlib.sha256,'sha512':hashlib.sha512}
def h(p,algo='sha256'): return HASHES.get(algo,HASHES['sha256'])(p.encode()).hexdigest()

def dict_attack(target, words:List[str], algo='sha256', cb=None)->Optional[str]:
    for i,w in enumerate(words,1):
        if h(w,algo)==target: cb and cb(i,len(words),f'Found {w}'); return w
        cb and cb(i,len(words),f'Testing {w}')
    return None

with tab1:
    pw=st.text_input("Enter password:", type="password")
    if pw: cat,sc,fb=check_strength(pw); st.write(f'**Strength:** {cat} ({sc}/8)')
    for f in fb: st.success(f) if f.startswith("✓") else st.warning(f) if f.startswith("■")else st.error(f)

with tab2:
    attack_type=st.radio("Type:",["Hash","Plaintext"])
    tgt=st.text_input("Target hash:" if attack_type=="Hash" else "Target password:", type="password")
    algo=st.selectbox("Algo",list(HASHES)) if attack_type=="Hash" else "sha256"
    file=st.file_uploader("Upload wordlist",type="txt")
    if file and st.button("Start Attack"):
        words=file.read().decode().splitlines()
        pb,txt=st.progress(0),st.empty()
        cb=lambda c,t,m: (pb.progress(c/t), txt.text(f'{m} {c}/{t}'))
        target=h(tgt) if attack_type=="Plaintext" else tgt
        res=dict_attack(target,words,algo,cb)
        st.session_state.found=res; st.success(f'☒ Found: {res}') if res else st.error("+ Not in dict")

with tab3:
    target=st.text_input("Target (brute force)",type="password")
    ml=st.slider("Max length",1,6,3); cs=st.text_input("Charset","abc123")
    if st.button("Start Brute Force"):
        pb,txt=st.progress(0),st.empty(); start=time.time()
        cb=lambda c,t,m:(pb.progress(c/t),txt.text(f'{m} {c}/{t} ETA: {(time.time()-start)/(c/t) - (time.time()-start):.1f}s'))
        res=brute_force(target,ml,cs,cb); st.session_state.found=res; st.success(f'☒ Found: {res}') if res else
st.error("+ Not found")

```

RESULT:

Navigation

Go to

Dictionary Attack

Brute-Force Attack

Password Strength Checker

Analyze Passwords File

Password Security Analyzer

A simple tool to demonstrate common password attacks and check password strength.

Dictionary Attack Simulation

A dictionary attack tries to crack a password by comparing it to a list of common words.

Select Attack Type:

Plaintext

Hash Value

Enter a plaintext password to attack:

password

Run Dictionary Attack

Dictionary Attack Simulation

A dictionary attack tries to crack a password by comparing it to a list of common words.

Select Attack Type:

Plaintext

Hash Value

Enter a plaintext password to attack:

uhsufdhudhjdfbnvbnd

Run Dictionary Attack

Attacking Plaintext

Searching for a dictionary word that matches the plaintext...

FAILURE!

 Password not found in the dictionary. (Time taken: 0.0 seconds)

Password Security Analyzer

A simple tool to demonstrate common password attacks and check password strength.

Password Strength Checker

This tool checks if a password is weak, medium, or strong based on its complexity.

Enter a password to check its strength:

password

Your password is Strong.

Great! This password is well-protected against common attacks.

DSPLab, AIML, DSCE DSCE 20251